


Politehnica University of Bucharest
Faculty of Automatic Control and Computers
Department of Automatic Control and Systems Engineering

BACHELOR THESIS

Synthesis of a Predictive Control Law for a Nonlinear System

Absolvent
Vener Andrei

Advisor
Prof. dr. ing. Florin Stoican

Bucharest, 2024

Contents

List of Figures

List of Tables

1 Introduction

The field of control systems is fundamental to numerous engineering applications, where maintaining stability and achieving desired performance are critical objectives. Among the various systems studied in control theory, the inverted pendulum stands out due to its inherent instability and nonlinear dynamics. This makes it an ideal candidate for testing and developing advanced control strategies.

I chose to study the inverted pendulum system because it provides a challenging yet insightful platform for exploring and demonstrating the capabilities of modern control techniques, particularly Model Predictive Control (MPC).

This thesis focuses on the synthesis and tuning of a predictive control law for the nonlinear dynamics of the inverted pendulum system. The primary objective is to implement and evaluate Model Predictive Control (MPC) for this purpose. MPC is a sophisticated control technique that optimizes control inputs by predicting the future behavior of the system over a finite time horizon. Unlike traditional control methods, MPC can handle multivariable control problems with constraints on both states and inputs, making it particularly suitable for the complex dynamics of the inverted pendulum.

The study begins with an in-depth modeling of the inverted pendulum system, capturing both its linear and nonlinear dynamics. The nonlinear model is derived using the Euler-Lagrange equations, providing a robust foundation for developing the MPC algorithm. This model serves as the basis for the predictive control strategy.

A contribution of this thesis is the implementation of Nonlinear Model Predictive Control (NMPC) using CasADi, a software framework designed for efficient evaluation of derivatives and large-scale optimization problems. The NMPC algorithm is tailored to handle the nonlinear dynamics of the inverted pendulum, ensuring accurate and efficient control in simulation.

A key contribution of this thesis is the design and construction of an inverted pendulum system specifically for testing the control algorithms developed in this study. The hardware setup features a custom-designed chassis, a stepper motor for precise control, and optical encoders for accurate measurement of the cart's position and the pendulum's angle. The system is powered by an ESP32 microcontroller, which interfaces with the sensors and executes the control algorithms.

Simulation and Experimental results demonstrate the effectiveness of the implemented MPC in maintaining the stability of the inverted pendulum. Various tests compare the performance of MPC with other control strategies such as Linear Quadratic Regulator (LQR) highlighting the advantages of MPC in handling complex, nonlinear systems.

In conclusion, this thesis presents a comprehensive study on the application of Model Predictive Control to a nonlinear and unstable system. The successful implementation and testing of the MPC algorithm on the inverted pendulum system underscore its potential for broader applications in advanced control engineering. Future work may explore the integration of more sophisticated algorithms and real-time optimization techniques to further enhance the system's performance.

2 Preliminaries

2.1 System Modeling of inverted pendulum

2.1.1 System description: Cart Pole

The inverted pendulum, specifically the cart-pole system 2.1, serves as the chosen testbed for evaluating Model Predictive Control (MPC). This system is a quintessential benchmark in control theory, frequently used to assess and compare the performance of various control algorithms due to its inherent instability and nonlinear dynamics.

While there are many variations, the cart-pole system considered hereafter comprises of a cart that can move horizontally along a single axis and a pendulum attached to the cart by a pivot joint. This joint is unactuated, meaning there is no direct control input applied to the pendulum itself. The primary control challenge lies in applying forces to the cart to keep the pendulum balanced upright.

By its nature, the cart-pole system is unstable. Without any external force applied to the cart, the pendulum will quickly deviate from its upright position and fall (since this corresponds to an unstable equilibrium point). This instability makes the system an excellent candidate for testing advanced control strategies. The goal of the control system is to maintain the pendulum in an upright position by moving the cart appropriately.

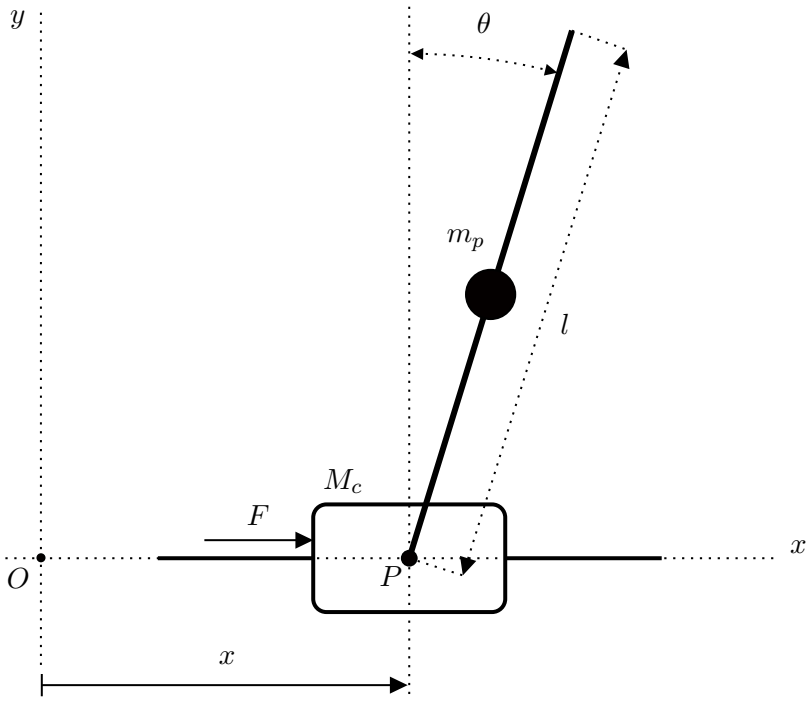


Figure 2.1: Cart pole system

2.1.2 Nonlinear model - The Euler-Lagrange equations

The rest of this section is inspired from [7]. To derive the mathematical model of the inverted pendulum, we use the Euler-Lagrange equations, which are a fundamental tool in classical mechanics. The starting point is the Lagrangian, defined as the difference between the kinetic energy (T) and the potential energy (V) of the system:

$$L \equiv T - V \tag{2.1}$$

where L is the Lagrangian, T is the kinetic energy, and V is the potential energy.

The Euler-Lagrange equations describe the dynamics of a system and are given by:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}}\right) - \frac{\partial L}{\partial q} = \frac{\partial W}{\partial q} \tag{2.2}$$

In this equation, W denotes the work done on the system, and q are the generalized coordinates. For the inverted pendulum, these coordinates are θ (the angle of the pendulum) and x (the horizontal position of the cart).

The kinetic energy (T) of the system can be expressed as the sum of the kinetic energies of the cart and the pendulum:

$$T = \frac{1}{2}M\dot{x}^2 + \frac{1}{2}mv_p^2 + \frac{1}{2}I\dot{\omega}^2, \tag{2.3}$$

where, M is the mass of the cart, m is the mass of the pendulum, v_p is the velocity of the pendulum, and I is the moment of inertia of the pendulum about its pivot.

The potential energy (V) of the pendulum, considering the height of its center of mass, is given by:

$$V = mgl\cos(\theta) \tag{2.4}$$

To determine the velocity of the pendulum (v_p), we decompose it into horizontal (v_{px}) and vertical (v_{py}) components:

$$v_p^2 = v_{px}^2 + v_{py}^2 = \left(\frac{d}{dt}\left(x + l\sin(\theta)\right)\right)^2 + \left(\frac{d}{dt}\left(-l\cos(\theta)\right)\right)^2 \tag{2.5}$$

By substituting v_p from (2.5) into (2.3), we get a more detailed expression for the kinetic energy:

$$T = \frac{1}{2}\dot{x}(M+m) + \frac{1}{2}ml^2\dot{\theta}^2 + m\dot{x}\dot{\theta}l\cos\theta + \frac{1}{2}I\dot{\theta}^2. \tag{2.6}$$

This comprehensive form accounts for the translational kinetic energy of the cart, the rotational kinetic energy of the pendulum, and the interaction between the translational motion of the cart and the rotational motion of the pendulum. Combining all these elements, we form the Lagrangian of the system:

$$L = \frac{1}{2}\dot{x}(M+m) + \frac{1}{2}ml^2\dot{\theta}^2 + m\dot{x}\dot{\theta}l\cos\theta + \frac{1}{2}I\dot{\theta}^2 - mgl\cos(\theta) \tag{2.7}$$

Using the Euler-Lagrange equation for the horizontal displacement x, we derive:

$$\begin{aligned} \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right) - \frac{\partial L}{\partial x} &= \frac{\partial W}{\partial x} \\ \ddot{x}(M+m) + m\ddot{\theta}l\cos\theta - ml\dot{\theta}^2\sin\theta &= F - b\dot{x} \end{aligned} \tag{2.8}$$

Here, F represents the external force applied to the cart, and b is the damping coefficient, which models friction acting on the cart.

Similarly, for the pendulum angle θ, we apply the Euler-Lagrange equation to obtain:

$$\begin{aligned} \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) - \frac{\partial L}{\partial \theta} &= \frac{\partial W}{\partial \theta} \\ \ddot{\theta}(ml^2 + I) + ml\ddot{x}\cos\theta - mgl\sin\theta &= -q\dot{\theta} \end{aligned} \tag{2.9}$$

In this equation, q is the damping coefficient which models the bearing friction of the pendulum. The resulting nonlinear equations of motion for the system are:

$$\ddot{x}(M+m) + m\ddot{\theta}l\cos\theta - ml\dot{\theta}^2\sin\theta = F - b\dot{x}, \tag{2.10a}$$

$$\ddot{\theta}(ml^2 + I) + ml\ddot{x}\cos\theta - mgl\sin\theta = -q\dot{\theta}. \tag{2.10b}$$

These nonlinear equations capture the coupled dynamics of the cart and pendulum system. They will be used to simulate the behavior of the system under Nonlinear Model Predictive Control (NMPC), which aims to predict and optimize the control actions over a future horizon to maintain stability and achieve desired performance.

2.1.3 Linearized Model - Force Input

To use Linear Quadratic Regulator (LQR) and compare its performance with Model Predictive Control (MPC), we need a linearized model of the system. This linearization is performed around the equilibrium point $(x, \dot{x}, \theta, \dot{\theta}) = (0, 0, 0, 0)$, which corresponds to the pendulum being in the upright position. By applying the small-angle approximation, which is valid when θ is close to zero, we can simplify the trigonometric functions as follows:

$$\begin{aligned} \cos\theta &\approx 1 \\ \sin\theta &\approx \theta \\ \dot{\theta}^2 &\approx 0 \end{aligned} \tag{2.11}$$

Substituting all the nonlinear parts from 2.10a,2.10b and using the small-angle approximation from 2.11 we arrive at

$$\ddot{x}(M+m) + ml\ddot{\theta} = F - b\dot{x} \tag{2.12}$$

$$\ddot{\theta}(ml^2 + I) + ml\ddot{x} - mgl\theta = -q\dot{\theta} \tag{2.13}$$

To make these equations suitable for state-space representation, we need to express the accelerations $\ddot{x}$ and $\ddot{\theta}$ in terms of the state variables x,$\dot{x}$,θ,$\dot{\theta}$ and the input force F. This involves rearranging the equations to isolate $\ddot{x}$ and $\ddot{\theta}$:

$$H = \frac{1}{Mml^2 + I(M+m)} \tag{2.14}$$

$$\ddot{\theta} = mlbH\dot{x} + mgl(M+m)H\theta - q(M+m)H\dot{\theta} - mlHF \tag{2.15}$$

$$\ddot{x} = -b(ml^2 + I)H\dot{x} - m^2l^2gH\theta + qmlH\dot{\theta} + (ml^2 + I)HF, \tag{2.16}$$

which corresponds to the space state system described by:

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \\ \mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \end{cases}$$

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -b(ml^2+I)H & -m^2l^2gH & qmlH \\ 0 & 0 & 0 & 1 \\ 0 & mlbH & mgl(M+m)H & -q(M+m)H \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ (ml^2+I)H \\ 0 \\ -mlH \end{bmatrix} F$$

$$y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} F$$

We modeled the system's dynamics to closely reflect real-world conditions. This linearized model accounts for various forces acting on the system, including the friction between the track and cart and the friction in the pendulum bearings. By incorporating these factors, we aim to achieve a high-fidelity representation of the actual physical forces impacting the system when the pendulum's angle is within the equilibrium point of the system. In this manuscript, we will use these system equations to compare model predictive control with other control algorithms.

2.1.4 Linearized model - Velocity Input

For the physical installation, a stepper motor is used to move the pendulum cart. As long as the cart's acceleration remains below the nominal torque of the stepper motor and the torque can overcome the friction forces between the track and the cart, we can assume that the speed of the stepper motor can be controlled with high accuracy.

Supposing that the cart-track friction and cart-pole bearing friction are small and ($b = 0, q = 0$), and the input of the system is the acceleration of the cart $u = \ddot{x} = \dot{v} = a$ the dynamics of the system can be further simplified. Substituting in the linearized model of the angle of the pendulum 2.13 we have:

$$\ddot{\theta}(ml^2 + I) + mlu - mgl\theta = 0. \tag{2.17}$$

As we have the acceleration of the cart as the input to our cart pole system, and we can control the position and velocity of the cart with good precision, we can ignore the linearized cart position equation 2.12. Using the 2.17 equation we have the state space system described by:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{mgl}{ml^2+I} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \\ -\frac{ml}{ml^2+I} \end{bmatrix} a$$

$$y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} a$$

The state space representation describes the dynamics of the system where x is the cart position, $\dot{x}$ is the cart velocity, θ is the pendulum angle, and $\dot{\theta}$ is the angular velocity of the pendulum. The input a is the acceleration of the cart, which we can control.

This linearized model allows for the implementation of Model Predictive Control (MPC) by predicting the future states of the system and optimizing the control inputs to achieve desired performance while adhering to system constraints.

Lastly, it should be said that the stepper motor while a good choice from the hardware viewpoint (due to its precision and high torque) has a nonlinear dynamic. The assumption that we have made (equating the input from the motor to be equal to the linear acceleration of the pendulum's cart) is a reasonable one for the range of velocities and accelerations at which we expect the pendulum to operate.

2.2 Model Predictive Control (MPC)

Model Predictive Control (MPC) is a model-based control technique used to predict future system behavior over a predefined finite horizon [14]. At any sampling instant, an MPC algorithm solves an optimization problem to determine the control inputs necessary at that moment, subject to constraints, to minimize a prespecified cost function. The first element of the optimal sequence is applied to the system's input, and the process is repeated at the next sampling instant, thus closing the loop.

Consequently, MPC has gained popularity in both industrial and academic fields for several vital reasons: one is its remarkable capability to handle multivariable constrained control systems, providing robustness and flexibility in solutions to highly complex processes. Furthermore, because MPC can predict the system's future behavior and act optimally based on that knowledge, it guarantees superior performance and efficiency. The drawback is that this method is, by definition, model-based. It's performance strongly depends on the equality of the model considered. If the model strongly deviates from the process (due to uncertainties, unmodelled perturbations and the like), then the prediction is no longer close to the reality and consequently the input resulted from the optimization fails to control adequately the closed-loop scheme.

Its wide adoption has also been driven by ease of implementation. User-friendly software tools and increased automation have rendered the deployment of MPC a much easier task for engineers and researchers. Additionally, recent advancements in cost-effective and powerful processing units now enable real-time computation for even large-scale systems. This level of computational efficiency makes real-time response to dynamic changes possible, ensuring accurate control and enhanced system stability.

2.2.1 MPC Mathematical formulation

The foundation of MPC lies in its ability to predict future system outputs based on a model and optimize the control inputs accordingly. This process involves solving a constrained optimization problem at each sampling time along the simulation/experiment. We may identify several steps/elements that appear in the MPC definition.

2.2.2 Prediction

MPC uses the system's mathematical model to predict the future states over a discrete prediction horizon. These predictions are based on the current state and potential control actions. The

system model can be represented as a constraint where:

$$\begin{aligned} x_{t+k+1} &= Ax_{t+k} + Bu_{t+k} && \text{Linear Model, State Space representation} \\ x_{t+k+1} &= f(x_{t+k}, u_{t+k}) && \text{Nonlinear Model} \end{aligned}$$

where x_k and u_k denote the state and control input at time k, respectively, and A and B are the system matrices. Note that the state is predicted, starting from the initial state (x_k, given as parameter to the algorithm) and a candidate sequence of inputs.

2.2.3 Optimization

The core of MPC is the optimization of a user-defined cost function over a defined prediction horizon N, which typically is the quadratic cost function that aims to minimize the tracking error and the control action:

$$\begin{gathered} q(x_k, u_k) = (x_k^{ref} - x_k)^T Q\ (x_k^{ref} - x_k) + u_k^T R u_k \\ \text{where} \quad Q \succeq 0, R \succeq 0 \end{gathered}$$

$$U_t^*(x(t)) := \arg\min_{U_t} \sum_{k=0}^{N-1} q(x_{t+k}, u_{t+k})$$

$$\begin{aligned} \text{subj. to} \quad & x_t = x(t) && \text{measurement} \\ & x_{t+k+1} = Ax_{t+k} + Bu_{t+k} && \text{system model} \\ & x_{t+k} \in \mathcal{X} && \text{state constraints} \\ & u_{t+k} \in \mathcal{U} && \text{input constraints} \\ & U_t = \{u_0, u_1, \ldots, u_{N-1}\} && \text{optimization variables} \end{aligned}$$

MPC operates on a receding horizon principle. This means only the first control action from the optimized sequence is implemented at each step. After applying the first control input, the horizon moves forward, and the process repeats.

Measurement The term *measurement* refers to the current state of the system, $x_t = x(t)$, which serves as the initial condition for the prediction model. Accurate measurement of the current state is crucial as it directly affects the accuracy of future state predictions and, consequently, the effectiveness of the control actions derived from the optimization process.

System Model The *system model*, defined by the equation $x_{t+k+1} = Ax_{t+k} + Bu_{t+k}$, describes the dynamics of the system where A and B are system matrices that relate the current state and control input to the next state. This model is essential for predicting future states under different control inputs and is a critical component in formulating the MPC problem.

State Constraints *State constraints*, denoted as $x_{t+k} \in \mathcal{X}$, specify the permissible limits or bounds within which the system states must remain. These constraints are often imposed to ensure operational safety, compliance with physical limitations, or other performance specifications.

Input Constraints Similarly, *input constraints* $u_{t+k} \in \mathcal{U}$ define the allowable range for control inputs. These constraints are typically set to prevent the actuator saturation and to protect the equipment from operating beyond its designed capabilities, thereby enhancing the reliability and safety of the system.

Optimization Variables The *optimization variables*, represented by $U_t = \{u_0, u_1, \ldots, u_{N-1}\}$, are the sequence of control inputs over the prediction horizon that the MPC algorithm optimizes. The choice of these variables directly determines the system's trajectory and must be chosen to optimize a predefined cost function while respecting all imposed constraints.

2.3 Comparison to other control algorithms

2.3.1 MPC vs. PID Control

PID (Proportional-Integral-Derivative) control is one of the most widely used control strategies due to its simplicity and ease of implementation. It adjusts the control input based on the current error, its integral, and its derivative. As such, it is a fair comparison with the MPC algorithm:

Pros of PID Simple design, easy to implement, and works well for single-variable systems with straightforward dynamics [1].

Cons of PID Tuning PID parameters can be challenging, especially for complex, nonlinear, or multivariable systems. PID control does not inherently handle constraints on inputs and states, making it less suitable for constrained environments [2].

Comparison with MPC While PID control is simpler and less computationally demanding, MPC offers superior performance in handling multivariable interactions and constraints. MPC optimizes control actions over a future horizon, making it more robust to disturbances and model inaccuracies [3].

2.3.2 MPC vs. Linear Quadratic Regulator (LQR)

LQR is an optimal control method that minimizes a quadratic cost function representing the trade-off between state deviations and control effort. It assumes a linear system model and provides a state feedback control law. In this sense, may be seen as the precursor to MPC which, in addition has a finite prediction horizon and considers explicitly constraints:

Pros of LQR Provides optimal control for linear systems with known parameters. It is systematic and can handle multi-variable systems [9].

Cons of LQR Assumes a linear model and does not explicitly handle constraints. Extensions to nonlinear systems or systems with constraints require additional methods [5].

Comparison with MPC While LQR is effective for linear systems without constraints, MPC extends this capability by incorporating constraints on inputs and states, which is crucial for real-world applications. MPC's flexibility allows it to handle a wider range of practical problems.

2.3.3 MPC vs. Robust Control

Robust control techniques, such as H-infinity control, are designed to maintain performance despite model uncertainties and external disturbances. These methods focus on guaranteeing stability and performance within specified bounds of uncertainty.

Pros of Robust Control Effective for systems with significant uncertainties. Provides guarantees on performance and stability [8].

Cons of Robust Control Can be conservative, leading to suboptimal performance in nominal conditions. The design process is more complex compared to PID or LQR [13].

Comparison with MPC Robust control and MPC both handle uncertainties, but MPC provides a more flexible framework by optimizing control actions based on future predictions. MPC can handle constraints and multivariable interactions more effectively than traditional robust control techniques [4].

2.3.4 Advanced MPC Techniques

Advanced MPC methods, such as robust MPC and nonlinear MPC, handle uncertainties and non-linearities in the system, offering improved performance and stability compared to traditional linear MPC methods (mainly because the model used in the prediction is closer to the plant dynamics, a linearization is always an approximation). These advanced techniques further extend the applicability of MPC to a broader range of complex systems, including those with significant non-linearities and model uncertainties [13, 3].

MPC's implementation complexity and computational demands are often cited as drawbacks compared to simpler controllers like PID. However, advancements in computational power and optimization algorithms have made MPC more accessible for real-time applications, even on resource-constrained devices such as embedded systems [6, 4]. Not least, approximate solutions, explicit formulations, each of them may help with the computation time.

To conclude, MPC offers superior performance for multivariable and constrained systems compared to PID, LQR, and robust control, albeit with higher computational demands. Advanced MPC techniques further enhance its applicability to a wider range of complex systems. Each control strategy has its own strengths and is best suited for specific applications depending on the system requirements and constraints.

3 MPC implementation

3.1 Nonlinear Model Predictive control using Casadi

Nonlinear Model Predictive Control (NMPC) is a sophisticated control technique that uses a nonlinear dynamic model of the system to predict and optimize control actions. The implementation of NMPC using CasADi involves leveraging a software framework designed for efficient and automatic evaluation of derivatives and solving large-scale optimization problems (the aforementioned CaADi). CasADi is widely used in control engineering, particularly for solving NMPC and moving horizon estimation (MHE) problems.

3.1.1 Defining the System Dynamics

To demonstrate the power of CasADi optimization, we will use the nonlinear equations 2.10b and 2.10a. Based on the discussion in this paper [7], since we simulate using a uniform solid bar, the center of mass is at the middle, which is $\hat{l}$. Substituting into the formula for the moment of inertia of a solid bar with uniform density, we have:

$$I = \frac{4}{3} m \hat{l}^2. \tag{3.1}$$

After substituting the moment of inertia into the nonlinear equations and modifying the equations to contain only lower derivatives, we obtain:

$$\ddot{\theta} = \frac{Mg\sin\theta - \cos\theta\left(f + m\hat{l}\dot{\theta}^2\sin\theta + F_f\right) - \frac{MM_f}{m\hat{l}}}{\frac{7}{3}M\hat{l} - m\hat{l}\cos^2\theta}, \tag{3.2a}$$

$$\ddot{x} = \frac{mg\sin\theta\cos\theta - \frac{7}{3}\left(f + m\hat{l}\dot{\theta}^2\sin\theta + F_f\right) - \frac{M_f\cos\theta}{\hat{l}}}{m\cos^2\theta - \frac{7}{3}M}, \tag{3.2b}$$

where the friction forces are given by:

$$M_f = \mu_p\dot{\theta} \tag{3.3a}$$

$$F_f = -\mu_c\dot{x} \tag{3.3b}$$

These equations are converted into Python code within the function `dynamics(x, u)`, where `x` is the state vector $\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]$ and `u` is the input force applied to the cart. The `dynamic` function is then used to compute the state derivatives based on the current state and control input, facilitating the implementation of the RK4 numerical integration method for simulating the system's nonlinear dynamics.

3.1.2 Simulating Nonlinear Dynamics

To simulate the nonlinear dynamics of our system accurately, we need a numerical integration method that can handle the complexity and nonlinearity of the equations.

The RK4 method is a fourth-order numerical integration technique that provides a good tradeoff between accuracy and computational cost. It calculates the next value of the state variable by taking the weighted average of four different estimates of the slope (derivative).

For a ordinary differential equation (ODE), defined as $\dot{x} = f(x, u)$, numerically sampling its associated trajectory with step size h, the state vector $\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]$ is updated using the following RK4 algorithm:

$$k_1 = f(\mathbf{x}_n, u) \tag{3.4}$$

$$k_2 = f\left(\mathbf{x}_n + \frac{h}{2}k_1, u\right) \tag{3.5}$$

$$k_3 = f\left(\mathbf{x}_n + \frac{h}{2}k_2, u\right) \tag{3.6}$$

$$k_4 = f(\mathbf{x}_n + hk_3, u) \tag{3.7}$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{3.8}$$

By using the RK4 method, we ensure that the nonlinear dynamics of the system are simulated with high accuracy, making it a robust choice for control and optimization tasks.

In other words, we may say that $\mathbf{x}_{n+1} \approx \int_{t_n}^{t_{n+1}=t_n+h} f(x(\tau), u)d\tau$.

3.1.3 Setting Up the Optimization Problem

A common cost function for the MPC problem is in quadratic form, defined over a finite horizon, with an additional terminal cost. The objective is to minimize this cost function while satisfying the state and input constraints.

$$\begin{aligned} \underset{\mathbf{u}_k}{\text{minimize}} \quad & \sum_{i=0}^{N-1} \left(\|\mathbf{x}_{i|k}\|_Q^2 + \|\mathbf{u}_{i|k}\|_R^2\right) + \|\mathbf{x}_{N|k}\|_P^2 \\ \text{subject to} \quad & \underline{u} \leq u_{i|k} \leq \overline{u}, \quad i = 0, \ldots, N-1 \\ & \underline{x} \leq x_{i|k} \leq \overline{x}, \quad i = 1, \ldots, N-1 \end{aligned}$$

To set up the optimization problem, we begin by creating an instance of the `Opti` class and defining the necessary parameters, variables, and constraints for the NMPC problem. This setup includes adding both input constraints and state constraints, which is facilitated by the powerful and flexible features of the CasADi library:

```
opti = casadi.Opti()
T = 20  # Seconds
N = 40  # Prediction Horizon
dt = 0.05  # Discretization time

# Parameters
opt_x0 = opti.parameter(4)  # initial state
ref = opti.parameter(4)  # reference state
```

```
# Control and State Variables that are going to be optimized
X = opti.variable(4, N+1)
U = opti.variable(1, N)  # Force
# Input Constraints
opti.subject_to(opti.bounded(-10, U, 10))
```

The core of the Model Predictive Control (MPC) problem involves ensuring that the optimized control inputs guide the plant simulation to follow the desired trajectory over the prediction horizon. To achieve this, we use the RK4 integration method and add constraints at each time step to enforce the system dynamics.

```
# Initial Conditions
opti.subject_to(X[:, 0] == opt_x0)
for k in range(0, N):
    x_next = rk4(X[:, k],U[0, k])
    opti.subject_to(X[:, k+1] == x_next)
```

We define the cost function to be minimized, which includes the state error and control effort:

```
# Cost matrices
Q = np.eye(4)
R = np.eye(1)
# Cost function
obj = 0
for i in range(N):
    err = X[:, i] - ref
    obj = obj + casadi.mtimes(casadi.mtimes(err.T, Q), err) + U[0, i] * R * U[0, i]
opti.minimize(obj)
```

Configuring the Solver and Initial States

We configure the solver settings and set the initial and final states:

```
opts_setting = {'ipopt.print_level': 0, 'print_time': 0,
'ipopt.acceptable_tol': 1e-8, 'ipopt.acceptable_obj_change_tol': 1e-6}
opti.solver('ipopt', opts_setting)

final_state = np.array([0, 0, 0, 0])  # Upright position
init_state = np.array([0, 0, np.pi, 0])  # Initial position

opti.set_value(ref, final_state)
```

3.1.4 Running the NMPC Loop

To execute the NMPC loop, we first initialize the state and control variables. Specifically, the *current_state* is initialized as a copy of *init_state* to avoid modifying the initial state. The control input array *u0* is initialized as a zero array with dimensions corresponding to the number of control steps N. Additionally, *next_states* is initialized as a zero array to store the predicted states. The iteration counter *mpciter* is also initialized and is incremented after each iteration.

We then iteratively solve the optimization problem until convergence or a maximum number of iterations is reached. The convergence criterion is based on the norm of the difference between the current state and the final desired state. Within the while loop, the current state is set as the initial condition for the optimization problem using *opti.set_value*. Initial guesses for the control inputs U and the state trajectory X are set using *opti.set_initial*. The optimization problem is then solved using *opti.solve*, and the optimized control inputs and state trajectory

are extracted from the solution. The current state is updated using the RK4 integrator with the optimized control input, and *u0* is updated to use the first control input of the optimized sequence for the next iteration. This process continues until the current state is sufficiently close to the final desired state or the maximum number of iterations is reached.

Starting from the initial state $[0, 0, -\pi, 0]$, which represents the inverted pendulum in the downward position, to the desired final state $[0, 0, 0, 0]$, the solver finds an input sequence that successfully swings up the pendulum to the upright position. The results are visualized in the following figures:

The figure 3.1 shows the trajectories of the cart position, cart velocity, pendulum angle, and pendulum angular velocity over time, demonstrating that the NMPC controller successfully guides the system to the desired state.

This figure 3.1 also shows the control input over time, illustrating that the NMPC respects the input constraints while achieving the desired control objectives.

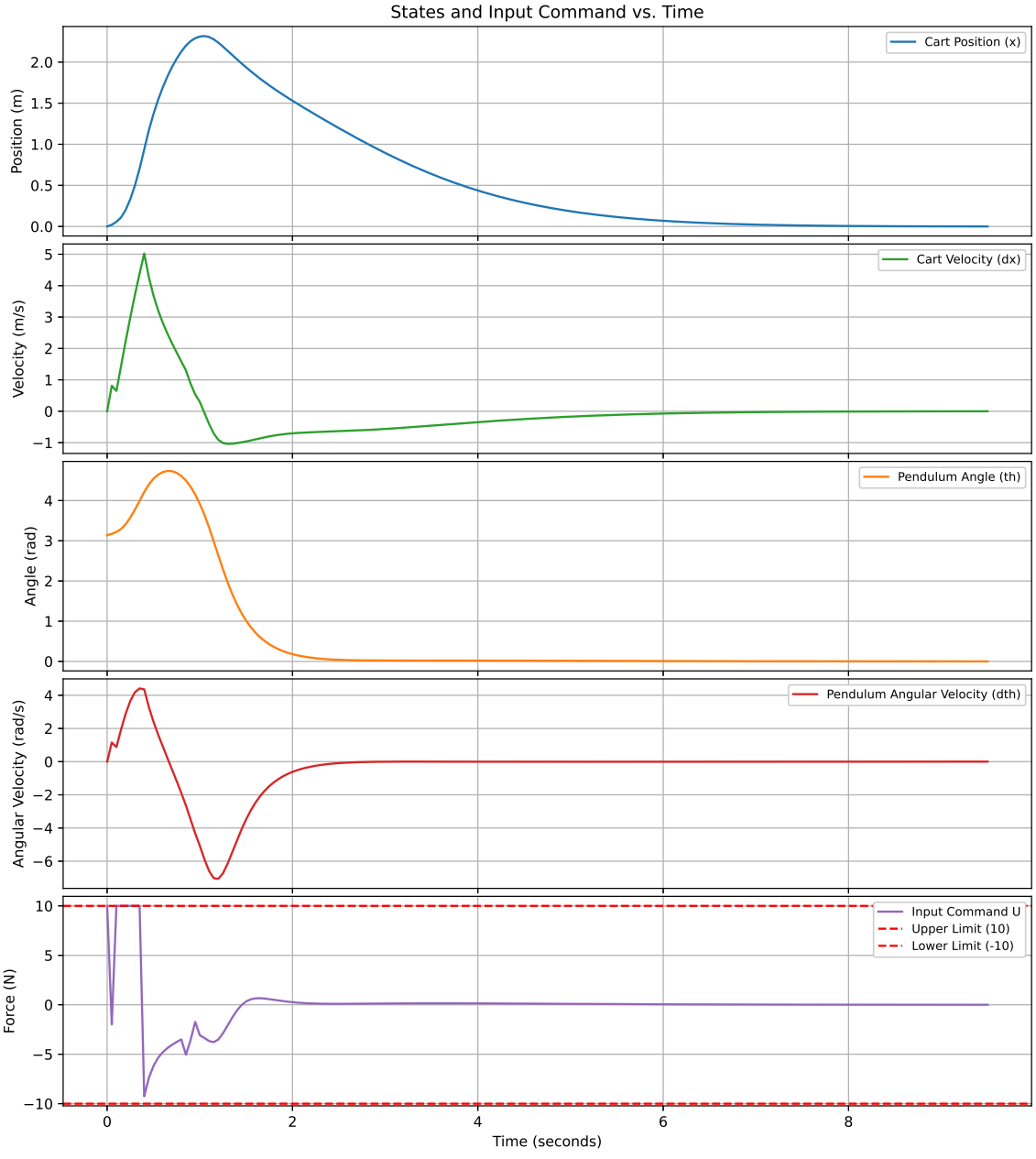


Figure 3.1: State trajectories and command of the NMPC-controlled system.

3.2 Comparison between LQR and MPC

The primary advantage of Model Predictive Control (MPC) lies in its predictive capabilities, enabling it to anticipate future states of the system. By leveraging knowledge of the reference signal within the prediction horizon, MPC minimizes the cost function while anticipating future changes in the reference. This predictive nature allows MPC to make informed control decisions, resulting in smoother and more precise control actions. In contrast, Linear Quadratic Regulator (LQR) cannot predict future reference changes and only responds after the reference signal has changed. This reactive approach often leads to overshoot and abrupt movements, as the controller must adjust to changes after they occur.

However, the implementation of MPC is significantly more complex compared to LQR. MPC requires solving an optimization problem at each time step, which demands substantial computational resources, particularly for large-scale systems. The complexity of MPC increases with the length of the prediction horizon and the number of constraints, making it a computationally intensive method.

On the other hand, LQR is easier to implement and computationally efficient. It involves solving a Riccati equation offline, and once the optimal gain matrix is determined, the control law is straightforward to apply. This simplicity makes LQR well-suited for systems with limited computational resources. However, LQR is designed for linear time-invariant systems and may not perform optimally if the system dynamics change. Additionally, LQR does not inherently handle constraints, which can be a significant limitation in practical applications.

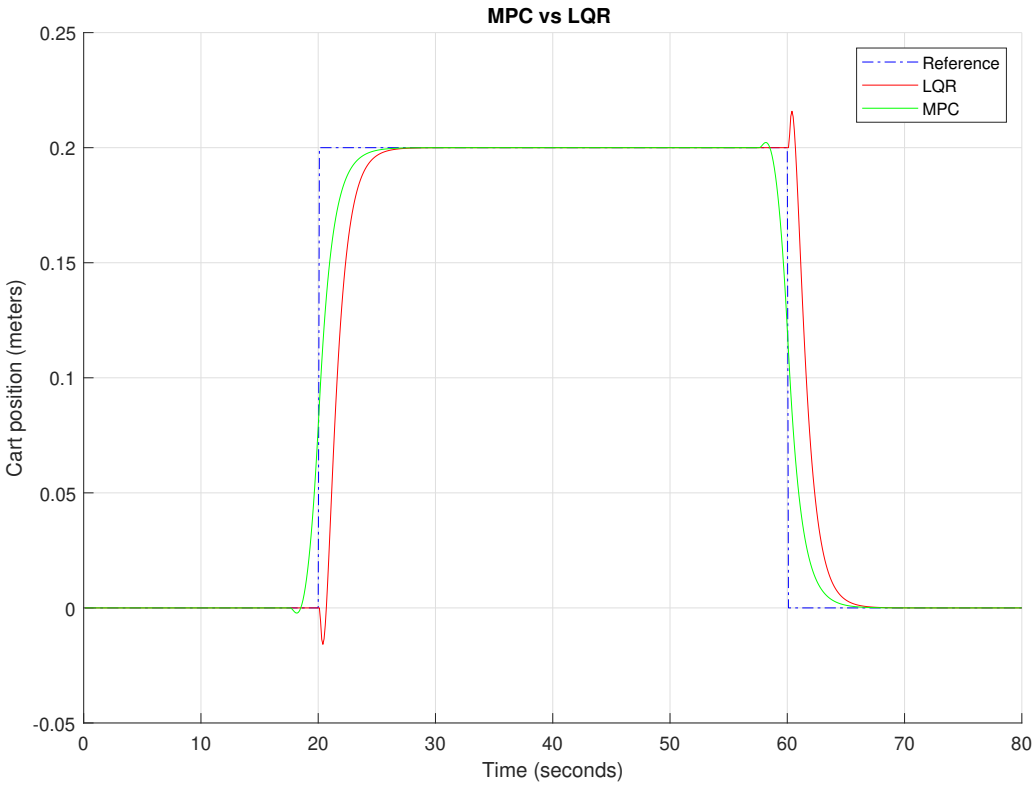


Figure 3.2: MPC vs LQR

To illustrate the performance differences between MPC and LQR, consider the step response of the cart's position to a reference signal, as shown in Figure 3.2. The reference signal (blue dashed line) represents the desired position of the cart, which starts at 0, jumps to 0.2 meters at around 20 seconds, remains constant until 60 seconds, and then returns to 0. The cart's

position when controlled by LQR is indicated by the red line, while the green line represents the cart's position under MPC.

The step response demonstrates that MPC achieves a smoother and more accurate tracking of the reference signal compared to LQR. The MPC-controlled system exhibits a slight overshoot at around 20 seconds but closely tracks the reference thereafter, settling accurately at the desired value with minimal steady-state error. Conversely, the LQR-controlled system shows more significant overshoot compared to MPC.

3.3 The Prediction Horizion and Tracking Error

To see how much the prediction horizon influence the tracking of a signal we compare the performance of Model Predictive Control (MPC) with different prediction horizons and Linear Quadratic Regulator (LQR) in tracking a sinusoidal reference signal for the position of the pendulum cart.

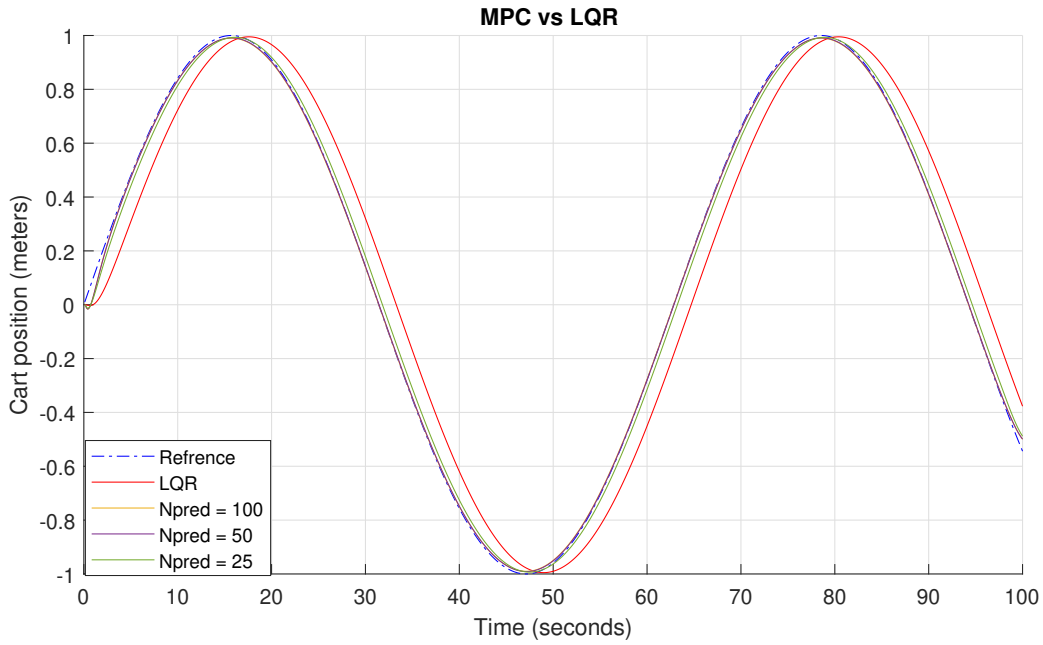


Figure 3.3: LQR vs MPC tracking

Control Algorithm	RMSE
MPC (Npred = 100)	0.01
MPC (Npred = 50)	0.0112
MPC (Npred = 25)	0.0303
LQR	0.1327

Table 3.1: Root Mean Square Error (RMSE) for Different Control Algorithms

The RMSE values indicate that the MPC controller with a prediction horizon (Npred) of 100 achieves the best tracking performance with an RMSE of 0.0113, outperforming all other tested configurations, including the LQR controller, which has the highest RMSE of 0.1327. While the MPC with Npred = 100 provides the most accurate tracking due to its ability to anticipate future reference changes, it is also the most computationally demanding. The slightly lower RMSE values for MPC with Npred = 50 (0.0112) and Npred = 25 (0.0303) show a trade-off between computational load and tracking accuracy, with shorter prediction horizons requiring less computational effort but resulting in slightly higher tracking errors.

3.4 Comparing Different Solvers and Prediction Times

To solve the optimization problem, there are numerous solvers available. In Figure 3.4, we compare the solvers from the CVX library (OSQP, SCS) and Ipopt from CasADi. **OSQP (Operator Splitting Quadratic Program)**[16] is an open-source solver for quadratic programming problems. It is designed to be efficient and scalable, making it suitable for real-time applications. **SCS (Splitting Conic Solver)**[11][10][12] is an open-source numerical optimization package for solving large-scale convex cone problems.

Solver	Horizon	Min Time	Max Time	Avg Time
OSQP	10	0.000000	0.016000	0.000959
OSQP	20	0.000993	0.042000	0.001415
OSQP	30	0.000998	0.039000	0.001992
OSQP	40	0.000992	0.059001	0.002388
OSQP	50	0.001777	0.079000	0.003242
SCS	10	0.000000	0.034998	0.000977
SCS	20	0.000000	0.029998	0.001458
SCS	30	0.000000	0.043001	0.001814
SCS	40	0.001778	0.056696	0.002890
SCS	50	0.002806	0.085486	0.005757
CasADi	50	0.005386	0.033001	0.006451

Table 3.2: Comparison of Solvers and Prediction Horizons

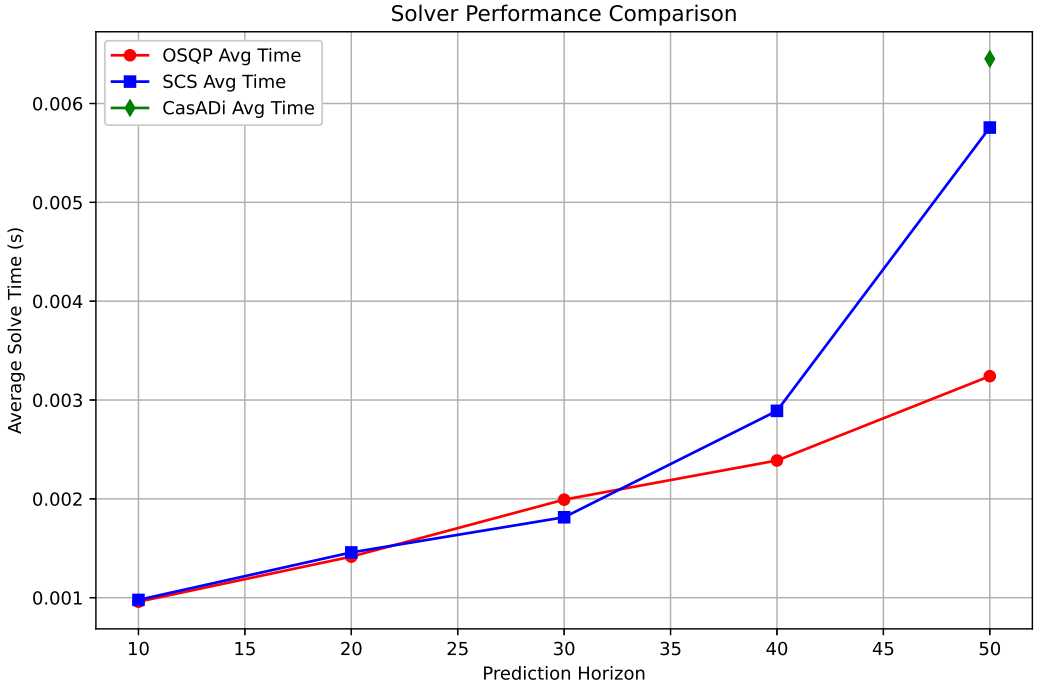


Figure 3.4: Comparison of solvers

We observe an exponential growth in computation time using the SCS solver, whereas the OSQP solver performed the best, being the fastest solver to find the optimal command. The CasADi solver was able to find a solution only with a prediction horizon of 50. This is because, with a smaller prediction horizon, the CasADi framework finds the problem to be infeasible and cannot find a solution.

4 Hardware Implementation and Results

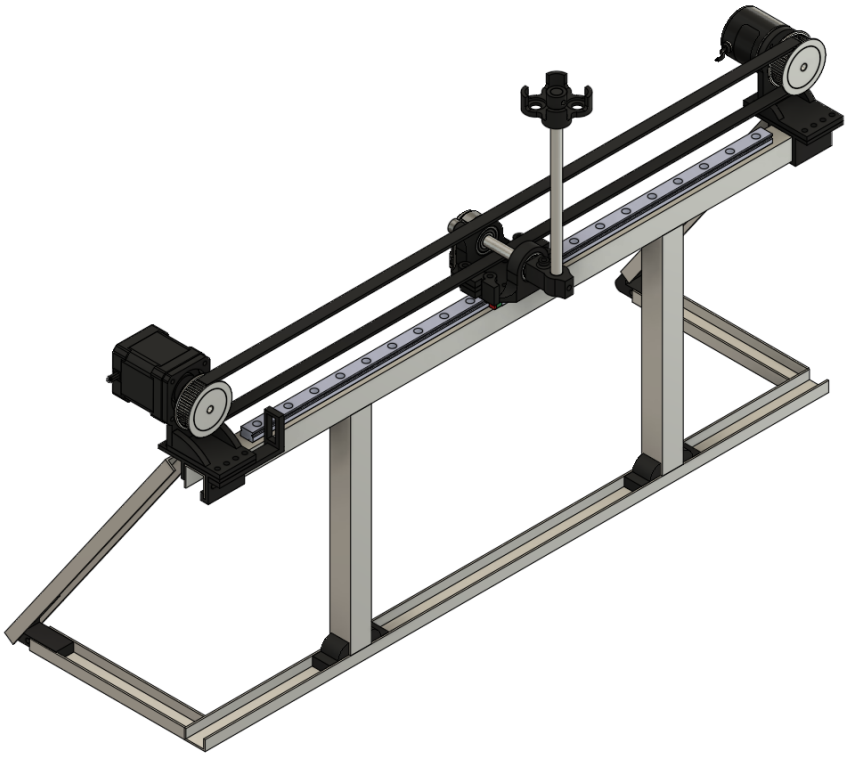


Figure 4.1: Top-Left View of Inverted Pendulum Model in Fusion 360

The hardware implementation of an inverted pendulum system is a critical aspect that ensures the system's stability and functionality. This chapter delves into the comprehensive design, construction, and integration of the system's hardware components. The primary objective of this project was to develop a low-cost, easily replicable cart-pole system using readily available materials and 3D printed components, thereby making it accessible to a wide range of users, including students and hobbyists.

The system was designed in Fusion 360, and all components were fabricated using an Ender 3 3D printer. The chassis is constructed from aluminum U-profile bars, connected with 3D printed parts, and assembled using M4 screws. The system utilizes a stepper motor and two optical encoders for precise control and measurement of the cart's position and the pendulum's angle.

Figure 4.1 illustrates the detailed design and dimensions of the constructed inverted pendulum system. This chapter provides an in-depth description of each hardware component and the overall assembly process.

4.1 Hardware parts Description

4.1.1 Chassis Design and Construction

The chassis of the inverted pendulum system forms the structural backbone, providing support and stability for all other components. The use of aluminum U-profile bars offers a balance between strength and weight, ensuring a robust yet lightweight frame. These bars were precisely cut to the required dimensions and connected using custom-designed 3D printed parts.

The 3D printed connectors were designed in Fusion 360 to ensure a perfect fit with the aluminum bars. These connectors facilitate a modular assembly, allowing for easy construction, disassembly, and modification. M4 screws are used to secure the connectors to the aluminum bars, providing a firm and stable connection.

This construction method not only reduces the overall cost but also simplifies the assembly process, making it accessible to individuals with basic tools and skills. The modular design ensures that the system can be easily transported and reassembled as needed.

4.1.2 Cart Mount

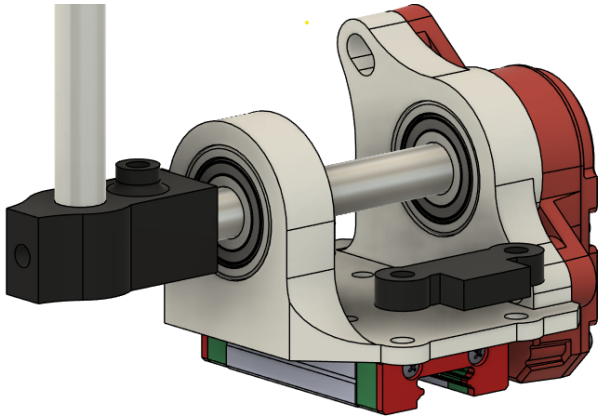


Figure 4.2: Close up view of the cart mount

The stepper motor is mounted on the chassis using a custom 3D-printed bracket, ensuring proper alignment and secure attachment. The motor drives a timing belt, which translates its rotational motion into linear motion of the cart.

To ensure movement along a single axis, an Mgn12h Mini Linear Rail Guide of 500mm was employed. This guide provides very smooth movement and can support high loads. The timing belt is tensioned by pulleys with a diameter of 37.7mm mounted on the stepper motor and the position optical encoder at either end of the track, ensuring precise and smooth movement of the cart.

The cart of the inverted pendulum is designed to attach seamlessly to the mount of the Mgn12h Mini Linear Rail guide. Perpendicular to the guide mount, there are two supports that house 608ZZ 8mm bearings. An aluminum bar traverses these bearings, connecting the pendulum to the cart and linking the angle encoder to the pendulum. The bearings and the aluminum bar support the entire weight of the pendulum, ensuring that the aluminum bar can rotate around a single axis, which is monitored by the optical angle encoder. This design provides a stable and precise connection, allowing for smooth and accurate measurement of the pendulum's angle.

A 3D-printed attachment connects the cart to the timing belt, ensuring a stable and reliable connection. This driving mechanism allows for precise control of the cart's position, which is essential for maintaining the balance of the pendulum.

4.1.3 Pendulum mount

The pendulum is the central component of the system, and its physical properties must remain consistent from run to run to accurately identify the system dynamics. To design a pendulum that is both easy to construct and allows for minor adjustments to the mass, I designed a 3D-printed part capable of holding up to four AA batteries. The batteries serve as the primary mass of the pendulum due to their widespread availability and consistent characteristics (mass and size). This support can be mounted on an 8mm aluminum bar, with the mounting position adjustable by loosening or tightening a screw. This design ensures that the pendulum's mass can be precisely controlled and adjusted as needed for accurate system dynamics analysis.

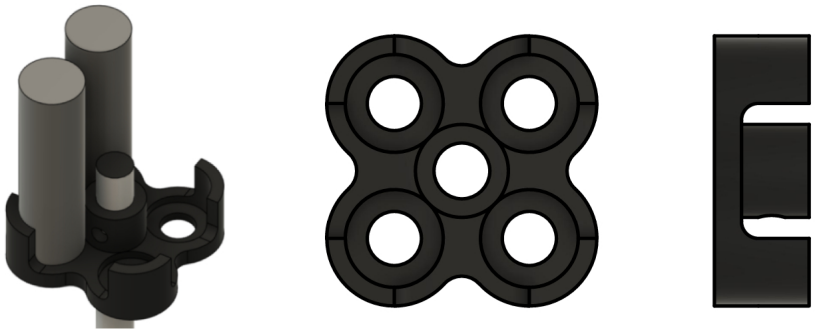


Figure 4.3: Close up view of the pendulum mount

4.2 Circuits Components Description

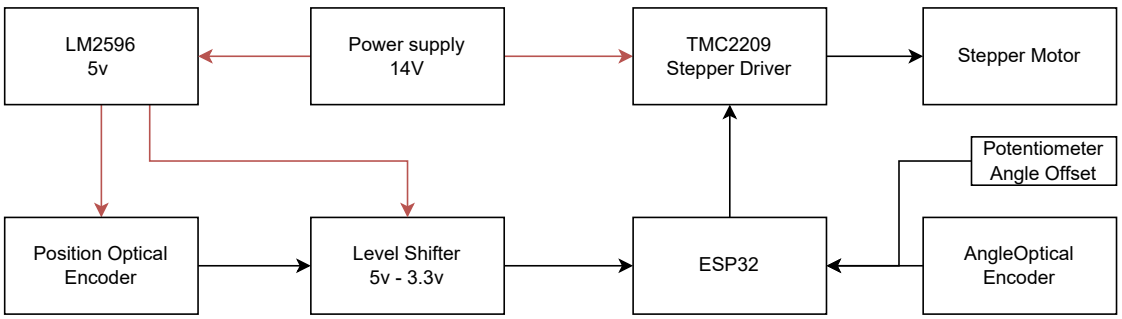


Figure 4.4: Circuit Components Connections

4.2.1 Power Supply and Level Shifter

The **Power Supply (14V)** is a crucial component, providing the necessary 14V DC to power various parts of the system. It serves as the primary power source for the entire setup. To accommodate components that require a 5V input, the 14V output from the power supply is stepped down to 5V using the **LM2596 Voltage Regulator (5V)**. This voltage regulation is essential for supplying the correct power to components such as the position optical encoder.

The interface between the 5V signal from the position optical encoder and the 3.3V logic level required by the ESP32 microcontroller is managed by a **Level Shifter** (5V to 3.3V). The level shifter ensures effective communication between components operating at different voltage levels, thereby maintaining the integrity and reliability of data transmission within the system.

4.2.2 Stepper Motor and Driver

A stepper motor was chosen for its ability to accurately control position and its high torque capabilities. The selected stepper motor is a NEMA 17 model, known for its widespread availability and reliable performance. It offers a resolution of 1.8 degrees per step, making it suitable for precise applications.

To drive the stepper motor, the TMC2209 driver was selected. This driver is a cost-effective solution that can control the stepper motor using two pins: STEP and DIR. The DIR pin determines the direction of the motor's rotation, controlled by setting the corresponding pin on the ESP32 to either a high or low logic level.

Whenever a rising pulse is detected on the STEP pin, the stepper driver commands the motor to move by one step. Therefore, the frequency of the pulses generated by the ESP32 directly controls the motor's velocity with high accuracy.

One drawback of stepper motors is that they can produce jerky motion at low speeds without microstepping. To address this issue, a microstepping resolution of 16 was chosen for the driver. This means that instead of a single step corresponding to 1.8 degrees, one step now corresponds to 0.1125 degrees. Given the diameter of the pulley, we can calculate the linear movement of the timing belt per rising pulse of the STEP pin as follows:

$$\frac{2\pi D_{\text{pulley}}}{400p} = 0.037 \text{ mm per step,}$$

$$\text{where } D_{\text{pulley}} = 37.7 \text{ mm and } p = 16 \text{ micro stepping resolution}$$

This resolution is sufficient to ensure smooth motion at low speeds. However, the stepper motor's mechanical limitations, such as torque drop-off at high speeds, must also be considered. As the step rate increases, the torque produced by the stepper motor decreases. This relationship means that at higher speeds, the motor may not have sufficient torque to drive the load effectively. To optimize performance, the system should be tuned to operate within the motor's optimal speed-torque curve.

Additionally, proper acceleration and deceleration profiles should be implemented to avoid losing steps or causing mechanical strain. The ESP32 can be programmed to ramp the step frequency up and down smoothly, ensuring that the motor operates efficiently within its capabilities. By carefully managing these parameters, the system can achieve both smooth low-speed motion and reliable high-speed performance.

4.2.3 Optical Encoders

Accurate measurement of the cart's position and the pendulum's angle is crucial for the control system. This is achieved using two optical encoders, described as follows.

4.2.4 Position Optical Encoder

The position optical encoder, model LPD3806-360BM[17], is mounted on the chassis to measure the linear position of the cart along the track. With a resolution of 600 pulses per revolution, it provides a resolution of feedback that is critical for the control algorithms implemented in the ESP32 microcontroller.

$$\frac{2\pi D_{\text{pulley}}}{1200} = 0.19 \text{ mm per enc pulse, where } D_{\text{pulley}} = 37.7 \text{ mm}$$

The encoder operates on a DC 5-24V power supply and can handle a maximum mechanical speed of 6000 revolutions per minute, with a response frequency of 0-20KHz. The encoder outputs AB two-phase quadrature rectangular pulses.

This precise positional data allows the ESP32 microcontroller to accurately determine the cart's position and make necessary adjustments to maintain the stability of the inverted pendulum

4.2.5 Angle Optical Encoder

The REV Through Bore Encoder REV-11-1271 [15] is designed to facilitate precise measurement of rotational position, with both relative and absolute position feedback through its ABI quadrature output and absolute position pulse output. It features a 1/2-inch Hex Through Bore and molded mounting holes for easy installation.

The encoder operates on an input voltage range of 3.3V to 5.0V and is compatible with 3.3V logic levels, making it suitable for direct interfacing with the ESP32 microcontroller. It provides a quadrature resolution of 2048 pulses per revolution.

$$\frac{2\pi}{2048} = 0.003 \; rad \text{ per count}$$

This high resolution ensures precise feedback necessary for the control algorithms in the ESP32 microcontroller to maintain the stability of the inverted pendulum. The encoder supports a maximum rotation speed of 10000 RPM and features a built-in magnet for incremental and absolute magnetic encoding. Its robust design includes factory-calibrated zero-position and a through-bore design for easy mounting on any shaft. The encoder also includes a JST-PH 6-pin connector for secure and reliable connections.

The REV Through Bore Encoder's precise angular data allows the ESP32 microcontroller to accurately determine the pendulum's angle and make necessary adjustments to maintain balance. This makes it an essential component for the system's accurate and stable performance.

4.2.6 Potentiometer Angle Offset

The potentiometer is an essential component used to manually adjust the angle offset of the inverted pendulum system. It provides a simple and effective means of fine-tuning the system's response to maintain the pendulum's balance.

In this system, the potentiometer is connected to the ESP32 microcontroller, which reads the analog voltage output from the potentiometer using its built-in Analog-to-Digital Converter (ADC). The position of the potentiometer's wiper determines the voltage level, which corresponds to a specific angle offset value. By adjusting the potentiometer, users can manually set the desired angle offset, allowing for precise control over the pendulum's equilibrium position.

The ability to adjust the angle offset is particularly useful during the calibration and tuning phases of the system. It enables users to compensate for any initial misalignment or biases in the pendulum's setup, ensuring that the control algorithms can effectively maintain balance. The ESP32 processes the potentiometer's input and integrates it into the control logic, dynamically adjusting the motor commands to achieve the desired angle.

4.2.7 ESP32 Microcontroller

The ESP32 microcontroller serves as the central processing unit of the inverted pendulum system, offering a robust and versatile platform for implementing control algorithms. The ESP32 is a low-cost, low-power system on a chip (SoC) with integrated Wi-Fi and dual-mode Bluetooth capabilities, making it suitable for a wide range of applications, including IoT and embedded systems.

The ESP32 features a dual-core Tensilica LX6 microprocessor with clock speeds up to 240 MHz, providing ample computational power for real-time control tasks. It includes 520 KB of SRAM, 4 MB of flash memory, and numerous peripheral interfaces, such as UART, SPI, I2C, and ADC, enabling seamless integration with various sensors and actuators.

In the inverted pendulum system, the ESP32 is responsible for processing data from the position and angle optical encoders. It receives quadrature signals from the encoders, calculates the cart's linear position and the pendulum's angular position, and executes control algorithms to maintain the pendulum's balance. The microcontroller generates precise control signals for the stepper motor driver (TMC2209), adjusting the cart's position to stabilize the pendulum.

The ESP32's built-in Wi-Fi capability allows for wireless monitoring and control of the system. This feature can be utilized for remote data logging, system diagnostics, and real-time adjustments, enhancing the system's flexibility and usability. Additionally, the microcontroller's Bluetooth capability can be used for local communication with other devices, facilitating seamless integration into larger control networks.

To interface the 5V signals from the position optical encoder with the ESP32's 3.3V logic level, a level shifter is employed. This ensures reliable communication between components operating at different voltage levels. The ESP32's ADC is used to read the output from the potentiometer, which allows for manual adjustments to the system's angle offset.

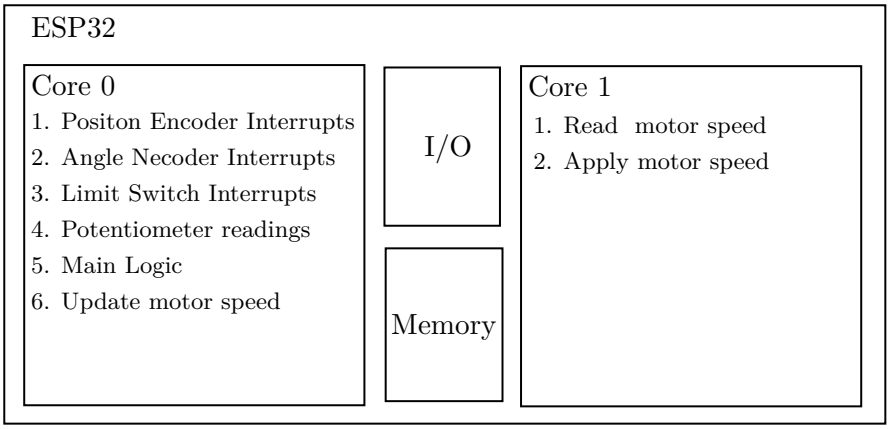


Figure 4.5: ESP32 parallel tasks

4.3 Utilizing Dual-Core Processing for Optimized Motor Control

To run the stepper motor at accurate speeds, we need to have absolute control over the frequency signal that is sent to the stepper driver. To accomplish this, one core (Core 1) of the two ESP32 cores is dedicated solely to handling the motor speed. This ensures that the frequency between the pulses remains consistent for a given speed, without being interrupted by other encoder interrupts or code logic.

The entire control algorithm and encoder interrupts are handled by Core 0. This parallelization ensures that the precise timing requirements for the stepper motor are maintained, while the computational load of the control algorithm and the handling of encoder signals are managed separately. By isolating these tasks on different cores, we prevent the control logic from impacting the motor speed control.

This code logic split shown in 4.5 allows for smoother and more reliable operation of the stepper motor, as the dedicated core can generate the necessary pulse frequencies without delays. It also enhances the responsiveness of the control algorithm, as Core 0 can process encoder inputs and compute control actions without competing for CPU time with the motor control tasks.

4.4 System Operation

The operation of the inverted pendulum system involves a coordinated process to maintain balance using discrete feedback and control. The ESP32 microcontroller is at the heart of this process, interfacing with the position and angle optical encoders to acquire real-time data.

Once the system is powered on, the ESP32 begins reading the quadrature signals from both encoders. The position optical encoder provides the linear displacement of the cart, while the angle optical encoder measures the pendulum's angular position. These readings are critical for the control algorithms that compute the necessary adjustments to keep the pendulum upright.

4.5 Output Data

After the successful hardware implementation of the inverted pendulum system, initial tests were conducted to collect fundamental readings of the system's behavior. The plots in Figure 4.6 display the system's responses over a 30-second period.

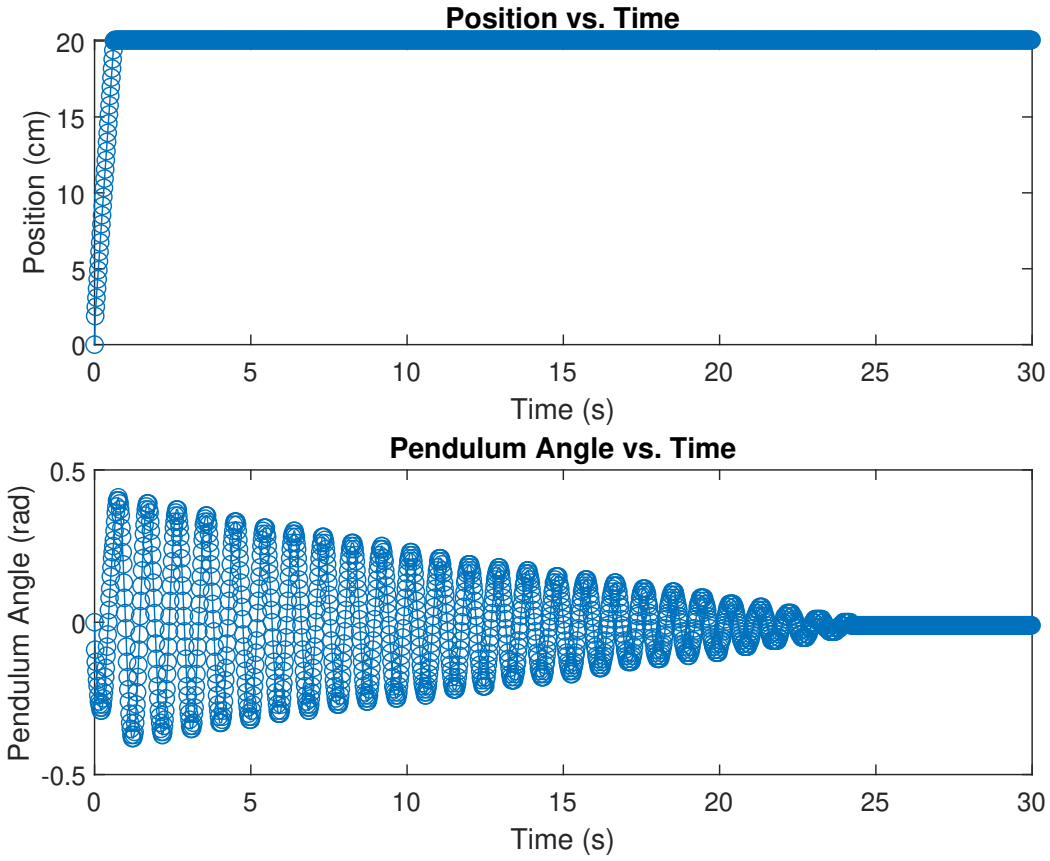


Figure 4.6: System response to position setpoint

The top plot depicts the cart's position as a function of time. Initially, the cart moves rapidly to a position of approximately 20 cm within the first few seconds and then stabilizes.

The bottom plot shows the pendulum's angle relative to the vertical axis over the same period. The pendulum begins with noticeable oscillations, which gradually diminish due to friction.

These basic readings provide an essential overview of the system's dynamic behavior, confirming that the hardware and motor commands implemented in the code are functioning as expected.

4.6 Velocity and Angle Estimation Using Discrete Software Filter

Accurately controlling the inverted pendulum system requires not only the position of the cart and the angle of the pendulum but also their respective velocities. However, direct velocity measurement is challenging due to sensor limitations and noise in the raw data. To overcome these challenges, a discrete software filter is employed to estimate the velocity of the cart and the angular velocity of the pendulum.

Another source of noise is the irregular availability of sensor data. When sensor readings occur too quickly, there can be instances where data is not available, causing the calculated velocities to temporarily drop to zero. As soon as data becomes available again, differentiating the latest encoder values can cause sudden spikes in the calculated velocity, which introduces jerky motion into the control algorithm. By using a discrete software filter, such as a low-pass filter, these issues can be mitigated, providing a smoother and more reliable signal for accurate velocity estimation.

The velocity of the cart can be estimated by differentiating the filtered position data with respect to time. Similarly, the angular velocity of the pendulum can be determined by differentiating the angular position data. Differentiation amplifies any noise present in the signal, which is why applying the filter after differentiation is important.

Mathematically, the velocity estimation can be described as follows:

$$v(t) = \frac{x(t) - x(t - \Delta t)}{\Delta t}$$

where $v(t)$ is the velocity at time t, $x(t)$ is the position at time t, and Δt is the time step between measurements.

Similarly, the angular velocity estimation can be described as:

$$\omega(t) = \frac{\theta(t) - \theta(t - \Delta t)}{\Delta t}$$

where $\omega(t)$ is the angular velocity at time t, $\theta(t)$ is the filtered angle at time t, and Δt is the time step between measurements.

To address the noise issue, a first-order low-pass filter with a cutoff frequency of 100 Hz was selected. The transfer function of the filter is given by:

$$H(s) = \frac{100}{s + 100}$$

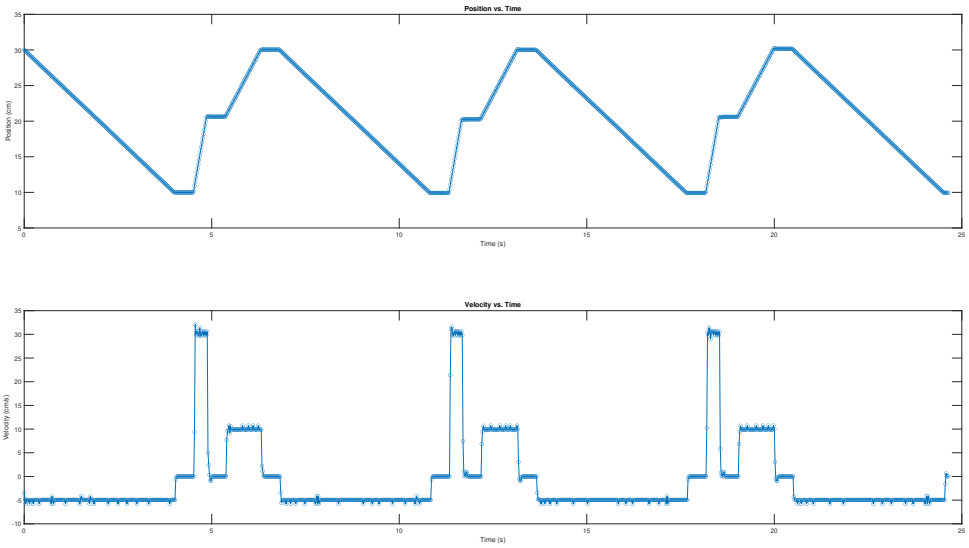


Figure 4.7: Velocity Setpoint Encoder Measurements

To test the effectiveness of this filter, the stepper motor was set to reach certain speeds, and the position encoder data was recorded. The filtered position data was then used to calculate

the speed, and the results were compared to the expected speed profiles. Figure 4.7 shows the results of these tests. The top plot illustrates the position of the cart over time, and the bottom plot shows the calculated velocity using the chosen low-pass filter.

As depicted in the plots, the filtered velocity data accurately matches the expected speed profiles. This confirms that the discrete software filter effectively smooths the velocity data. These results validate the use of the filter for precise control of the inverted pendulum system, ensuring that the hardware and control algorithms function as intended.

Using the same filter for angular velocity estimation, we now have access to all the states of the pendulum: cart position, cart velocity, pendulum angle, and angular velocity. With all these states available, we can now test various control algorithms on our inverted pendulum hardware.

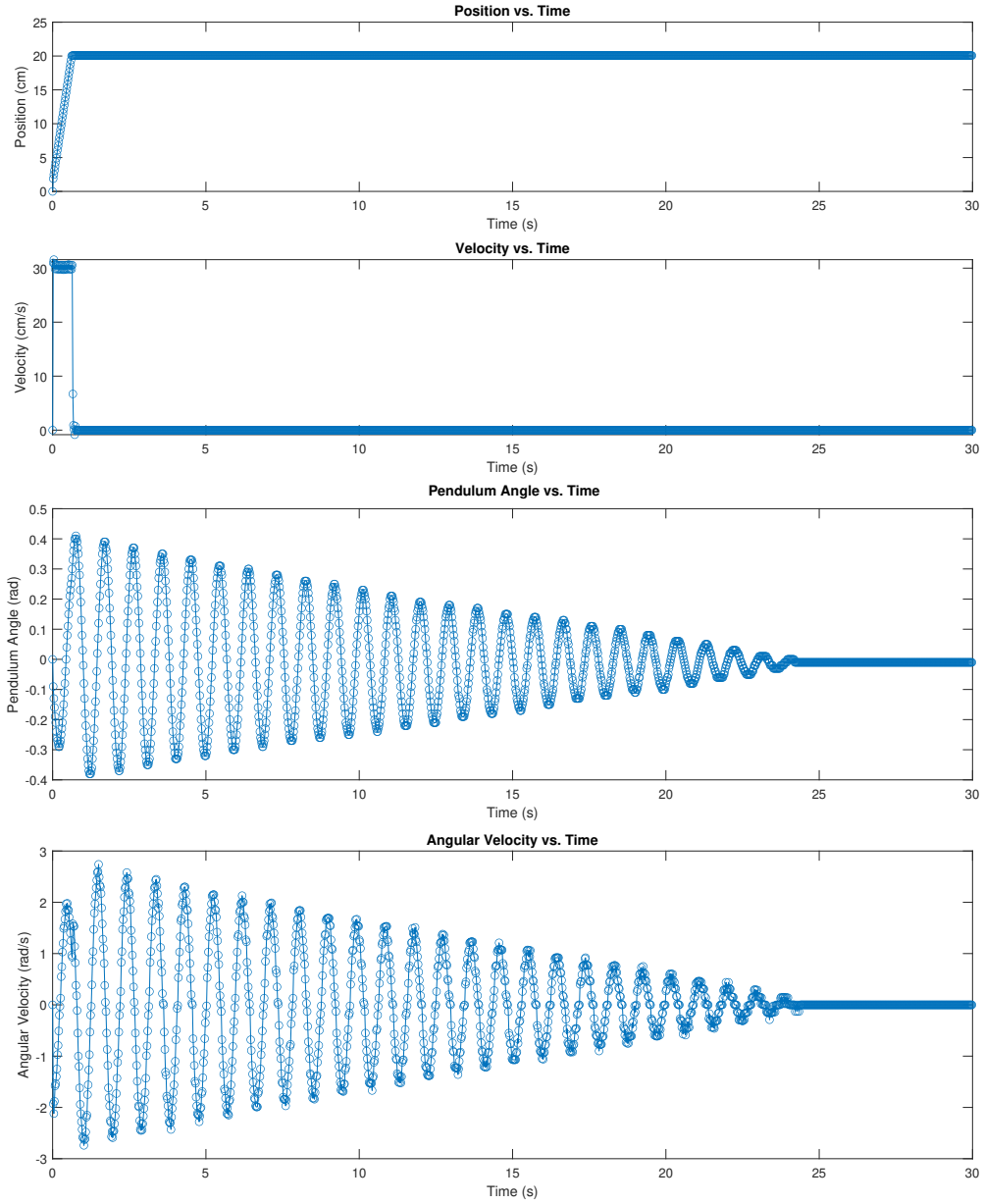


Figure 4.8: Velocity Setpoint Encoder Measurements

4.7 Finding the maximum acceleration/speed

Determining the maximum acceleration and speed of the stepper motor is crucial for ensuring the stability and performance of the inverted pendulum system because these are in fact the bounds that we have to consider (either as saturation for LQR or, explicitly in the MPC algorithm). Knowing these limits helps in designing effective control algorithms that can operate within the motor's capabilities without causing it to stall or lose steps.

To identify the maximum acceleration and speed, we conducted a series of tests where the speed of the stepper motor was incrementally increased from 5 cm/s to 60 cm/s. The objective was to observe if the stepper motor could accelerate from 0 to the desired speed quickly and maintain the required torque throughout the acceleration phase.

During the tests, the stepper motor successfully accelerated to the target speeds up to approximately 55 cm/s. Beyond this point, the motor struggled to maintain the desired speed, indicating that the maximum reliable speed and acceleration for our system is around 55 cm/s. To ensure consistent performance and avoid pushing the motor to its absolute limits, we chose 50 cm/s as the maximum operational speed for our system.

These findings are essential for defining the operational parameters of the inverted pendulum system, ensuring that the motor performs reliably within its capabilities. By understanding these limitations, we can optimize the control algorithms to maintain stability and responsiveness without exceeding the motor's performance envelope.

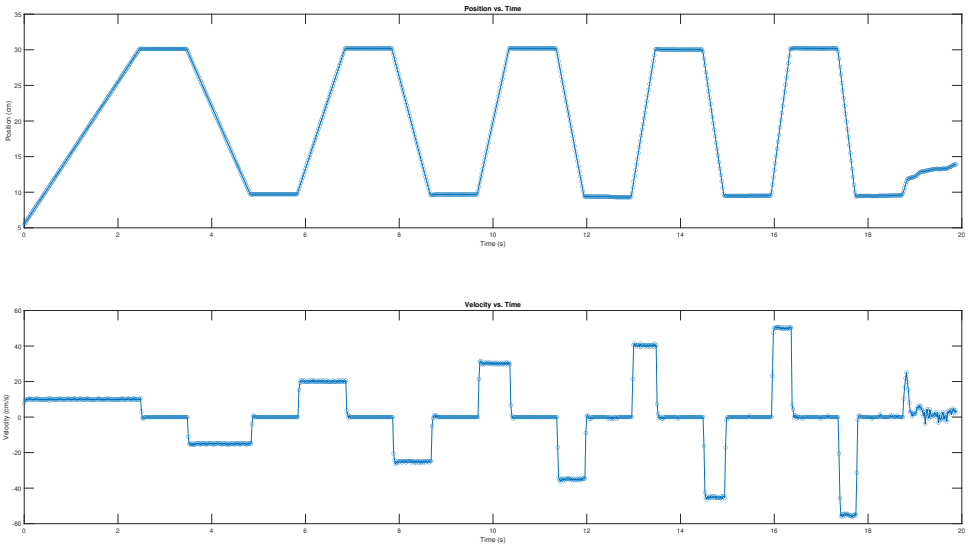


Figure 4.9: Maximum acceleration tests Max ACC = 50 cm/s^2

4.8 LQR Control

The plots presented in Figure 4.10 illustrate the system's response under Linear Quadratic Regulator (LQR) control with the following weight matrices: $Q = \text{diag}([50, 1, 10, 1])$ and I_1. The goal of this control setup is to bring all the state variables of the inverted pendulum system to their reference values, which are zero for position, velocity, angle, and angular velocity.

The top plot shows the cart's position over time. Initially, the cart exhibits significant oscillations, but these oscillations gradually decrease as the control system works to stabilize

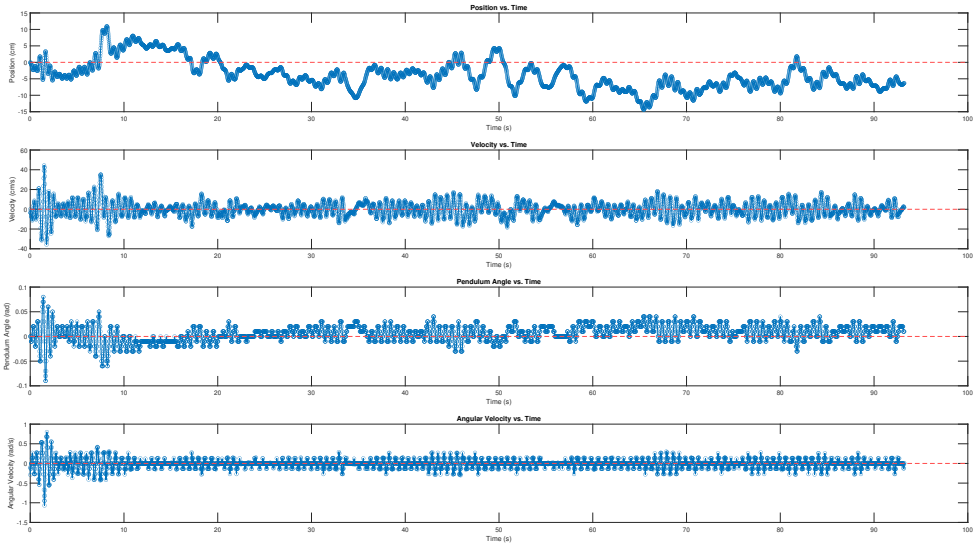


Figure 4.10: System response under LQR control with $Q = \text{diag}([50, 1, 10, 1])$ and $R = 1 \cdot \text{eye}(1)$

the position around the zero reference. Despite some residual oscillations, the general trend indicates a movement towards the desired position.

The second plot depicts the cart's velocity over time. Similar to the position plot, the velocity starts with large oscillations which diminish over time. The oscillatory behavior is a result of the control system's efforts to correct deviations from the reference state. The presence of high-frequency noise in the velocity signal suggests that there may be model uncertainties or unmodeled dynamics affecting the system.

The third plot illustrates the pendulum's angle relative to the vertical axis. The angle starts with noticeable oscillations, which decrease as the system stabilizes. Although the pendulum angle does not perfectly converge to zero, it tends to hover around the reference state, indicating that the LQR controller is effectively reducing the angle deviations.

The bottom plot shows the angular velocity of the pendulum. The initial large fluctuations are gradually damped, and the angular velocity trends towards zero. However, similar to the velocity plot, some high-frequency noise persists, which could be attributed to model inaccuracies or external disturbances.

Overall, the readings show that all state variables tend to move towards their reference values of zero. The remaining noise and tracking error are likely due to model uncertainties and the limitations of the system modeling. The LQR controller effectively stabilizes the system, but further refinement of the model and control parameters could help reduce the residual oscillations and noise, leading to more precise control performance.

4.9 MPC Control

Running the Model Predictive Control (MPC) algorithm directly on the ESP32 microcontroller would not be feasible due to its limited computational resources and the stringent time constraints required to find a solution within the sampling time. To address this limitation, we implemented a hybrid approach where the state of the system is read by the ESP32 and then transmitted via Serial communication to a Python script running on a more powerful computer.

The Python script uses the OSQP solver from CVX, chosen for its speed and efficiency as demonstrated in our solver comparison tests (refer to Table 3.2). Once the optimal control solution is computed, the command is sent back to the ESP32 to be applied. The entire process, including solver computation and serial communication, is completed within 0.02 seconds, matching the sampling time of the system.

Figure 4.12 shows the real-time states acquired from the system during MPC operation. As observed, the MPC algorithm manages to stabilize the inverted pendulum, but there are notable oscillations in both the position and the command signals. These oscillations may stem from model uncertainties, particularly those associated with the complex mechanics of the stepper motor. Specifically, there appears to be a discrepancy between the commanded speed of the cart and the actual measured data.

Figure 4.11 illustrates the control commands generated by the MPC. Despite the stabilization achieved, the presence of large oscillations in the command signal suggests areas for improvement. These could include refining the model to better capture the dynamics of the stepper motor and the cart, or tuning the MPC parameters to reduce sensitivity to such uncertainties.

The system's response was observed and analyzed to verify the effectiveness and compliance with the specified constraints. We observed that the control algorithm adhered to the specified constraints, ensuring that the command did not exceed $\pm 5\,\mathrm{m/s^2}$.

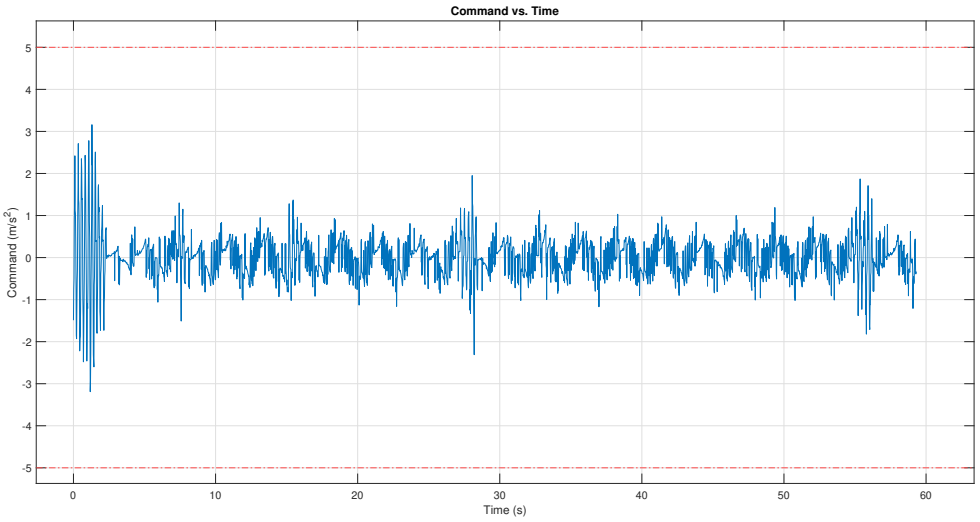


Figure 4.11: Online MPC command

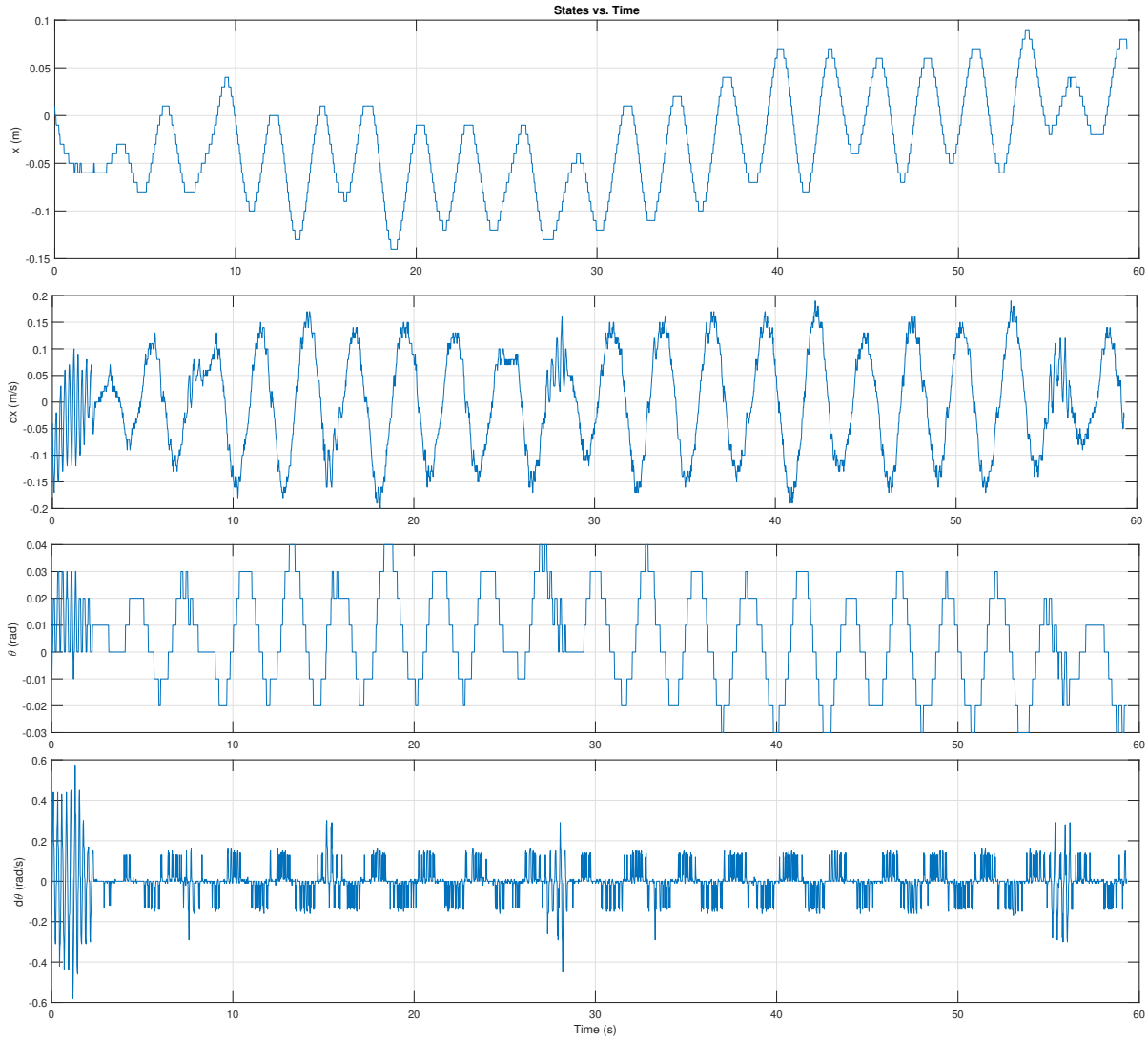


Figure 4.12: Online MPC states

5 Conclusions & Future Work

In this thesis, we designed and built our own inverted pendulum system and explored various methods to implement efficient control algorithms to stabilize it. This hands-on approach provided valuable insights into the processes behind implementing a control system, including modeling, simulation, hardware integration, and algorithm optimization.

One of the key takeaways from this work is the significant computational demand of Online Model Predictive Control (MPC). The computational load sometimes exceeds the chosen sampling time of 0.02 seconds, highlighting the need for improvements in both hardware and software aspects of the system. Enhancing the communication between the ESP32 microcontroller and the PC is crucial, as the current serial communication method is too slow, reducing the available time to compute the optimal solution.

A potential alternative is Explicit MPC, which pre-computes the control law offline. While this method can significantly reduce online computation time, it introduces the challenge of managing a large number of critical zones. Efficiently searching through these zones to ensure lookup times remain under 0.02 seconds is a necessary area of optimization.

Improving system identification is another critical aspect for future work. The current approximation of the stepper motor speed as the system input is not sufficiently accurate, and the system exhibits complex nonlinear behaviors that are not fully captured in the current model. Developing a more accurate and comprehensive system model will be essential for enhancing control performance.

Future work could also explore the integration of more sophisticated algorithms and real-time optimization techniques. For instance, Offset free MPC[13] could be investigated to handle model uncertainties and external disturbances more effectively. Additionally, leveraging more powerful microcontrollers or dedicated hardware for real-time computation could further improve the system's responsiveness and stability.

Another interesting direction for future research is the application of machine learning techniques to enhance control strategies. For example, reinforcement learning could be used to develop control policies that improve over time based on the system's performance.

In conclusion, this thesis has demonstrated the practical challenges and potential solutions in implementing MPC for an inverted pendulum system. The experience gained from this project provides a solid foundation for further research and development in advanced control systems. By addressing the identified challenges and exploring new techniques, future work can significantly enhance the performance and applicability of control algorithms for complex, nonlinear systems like the inverted pendulum.

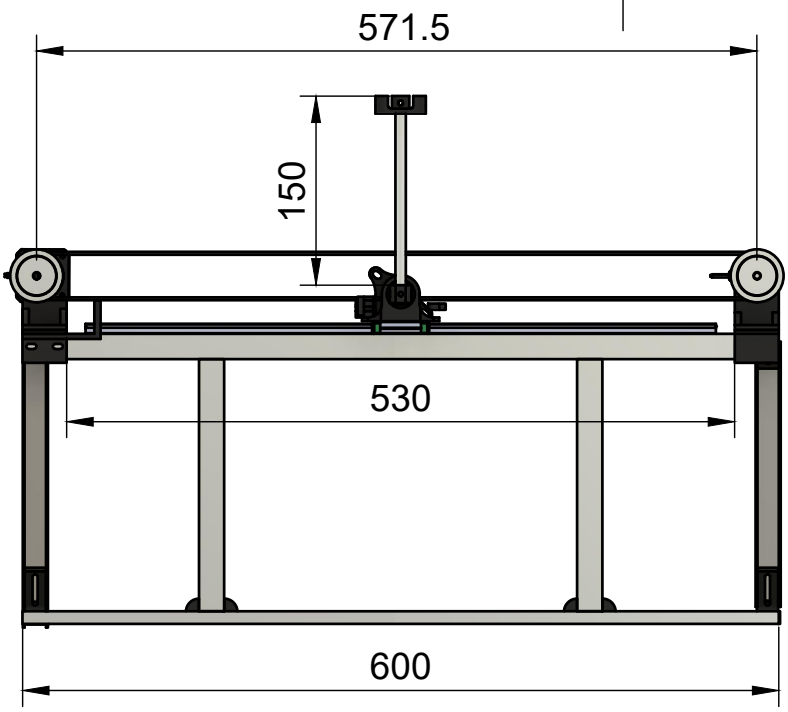
6 Personal contribution

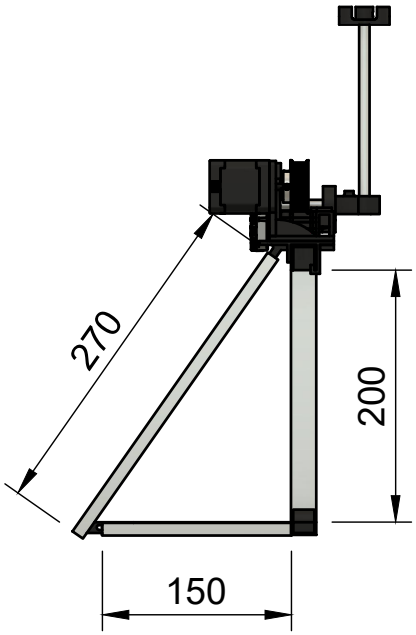
The next table describes the personal contribution presented in this bachelor thesis.

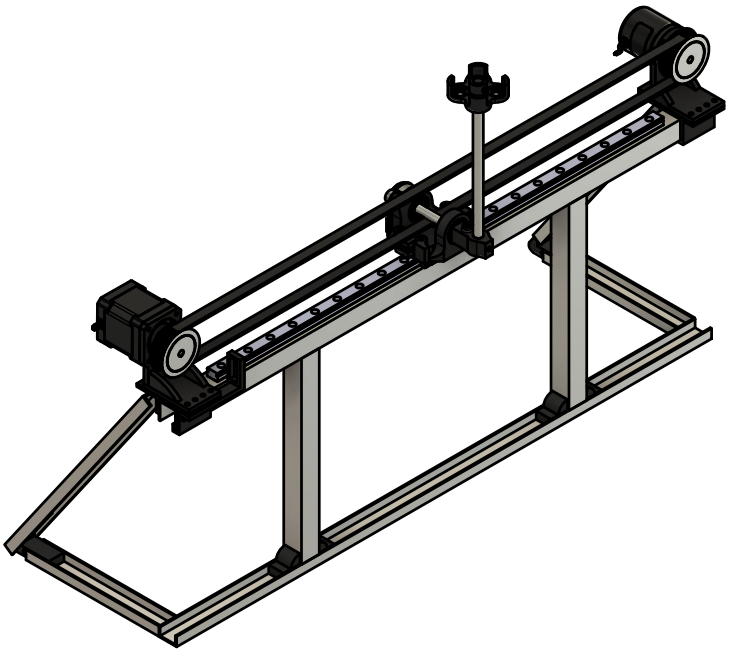
Synthesis of a Predictive Control Law for a Nonlinear System **Vener Andrei** **Florin Stoican**		
	Activity	Duration [Days]
1	Research of MPC	7
2	Implementation of linear MPC Matlab	7
3	Implementation of non-linear MPC in Python and Casadi	14
4	Hardware Design and 3D Printing	28
5	Hardware assembly and circuits connections	7
6	Hardware Testing	2
7	LQR Code and Hardware Testing	10
8	MPC Code and Hardware Testing	5
9	Writing of thesis	14

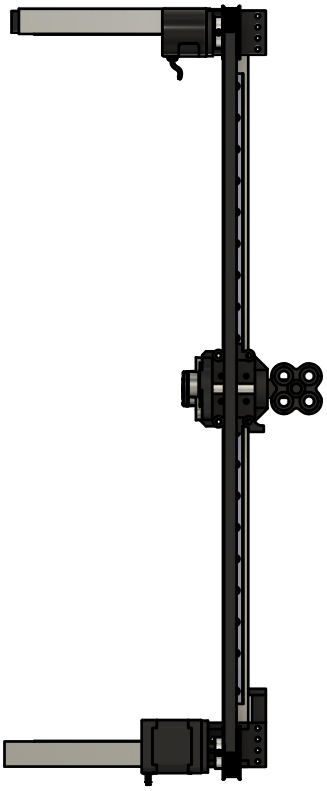
Table 6.1: Personal Contribution

Appendices









Dept.	Technical reference	Created by Vener Andrei 6/8/2024	Approved by
		Document type	Document status
		Title InvertedPendulum	DWG No.
			Rev. / Date of issue / Sheet 1/1

Bibliography

[1] Ahmed A Abdelrauf, M Abdel-Geliel, and E Zakzouk. "Adaptive PID controller based on model predictive control". In: *2016 European Control Conference (ECC)*. IEEE. 2016, pp. 746–751.

[2] A Aboelhassan et al. "Design and Implementation of Model Predictive Control Based PID Controller for Industrial Applications". In: *Energies* 13.24 (2020), p. 6594.

[3] R Balan et al. "Nonlinear Control Using a Model Based Predictive Control Algorithm". In: *2007 International Symposium on Computational Intelligence in Robotics and Automation*. IEEE. 2007, pp. 510–515.

[4] Ján Drgoňa et al. "Approximate model predictive building control via machine learning". In: *Applied Energy* 218 (2018), pp. 199–216.

[5] Mostafa Al-Gabalawy, N S Hosny, and Abdel-hamid S Aborisha. "Model predictive control for a basic adaptive cruise control". In: *International Journal of Dynamics and Control* (2021), pp. 1–12.

[6] Luis García, Alejandro Astudillo, and Esteban Rosero. "Fast Model Predictive Control on a Smartphone-based Flight Controller". In: *2019 IEEE 4th Colombian Conference on Automatic Control (CCAC)*. IEEE. 2019, pp. 1–6.

[7] Colin D. Green. "Equations of Motion for the Cart and Pole Control Task". In: (2024). URL: https://sharpneat.sourceforge.io/research/cart-pole/cart-pole-equations.html.

[8] R Marco, D Semino, and A Brambilla. "From Linear to Nonlinear Model Predictive Control: Comparison of Different Algorithms". In: *Industrial & Engineering Chemistry Research* 36.6 (1997), pp. 1708–1716.

[9] Kenneth Muske and James B Rawlings. "Model predictive control with linear models". In: *AIChE Journal* 39.2 (1993), pp. 262–287.

[10] Brendan O'Donoghue. "Operator Splitting for a Homogeneous Embedding of the Linear Complementarity Problem". In: *SIAM Journal on Optimization* 31 (3 Aug. 2021), pp. 1999–2023.

[11] Brendan O'Donoghue et al. "Conic Optimization via Operator Splitting and Homogeneous Self-Dual Embedding". In: *Journal of Optimization Theory and Applications* 169.3 (June 2016), pp. 1042–1068. URL: http://stanford.edu/~boyd/papers/scs.html.

[12] Brendan O'Donoghue et al. *SCS: Splitting Conic Solver, version 3.2.4*. https://github.com/cvxgrp/scs. Nov. 2023.

[13] G Pannocchia. "Robust model predictive control with guaranteed setpoint tracking". In: *Journal of Process Control* 14.8 (2004), pp. 927–937.

[14] Sasa V Rakovic and William S Levine. "Handbook of model predictive control". In: (2018).

[15] *REV-11-1271 Data Sheet*. https://docs.revrobotics.com/through-bore-encoder/specifications. Nov. 2023.

[16] Bartolomeo Stellato et al. "OSQP: An operator splitting solver for quadratic programs". In: *Mathematical Programming Computation* 12.4 (2020), pp. 637–672.

[17] *The position optical encoder, model 600P/R Incremental Rotary Encoder*. https://www.alldatasheet.com/datasheet-pdf/pdf/1150705/ETC2/LPD3806-360BM.html. Nov. 2023.